

Hierarchical Abstract Trees for Cross-Document Retrieval-Augmented Generation with CUDA-Accelerated GPU Computing

Dr. Kaushalya Thopate¹, Nivesh Jain¹, Hasan Rupawalla¹, Harsh Nayak¹, Rachit Ingole¹

© The Authors 2026. This paper is published with open access at SuperFri.org

Retrieval-augmented generation (RAG) systems commonly retrieve passages from a flat vector index. This design is simple, but it evaluates a large number of unrelated passages and offers limited context for questions whose evidence is distributed across documents. This paper presents HAT-RAG, a hierarchical abstract tree for cross-document retrieval-augmented generation. The system constructs leaf passages, local cluster abstracts, and higher-level root abstracts, then performs top-down similarity search with branch pruning. Batch cosine similarity is implemented with PyTorch tensors and can execute on NVIDIA CUDA devices. The repository also includes flat, graph-inspired, collapsed-tree, and hybrid baselines, together with an evaluator that records latency, evaluated nodes, document coverage, and tail latency. The current implementation and its reproducible software artifacts are described, while reported performance values are identified as repository-level measurements that require confirmation on a fixed benchmark and hardware configuration.

Keywords: retrieval-augmented generation, hierarchical retrieval, cross-document question answering, CUDA, GPU computing, vector search.

Introduction

Retrieval-augmented generation combines a language model with an external evidence collection and is widely used for knowledge-intensive question answering [5, 11]. Dense passage retrieval provides an effective baseline for locating semantically related passages [10], but a flat search space treats every passage as an independent candidate. This can increase search work and can return fragmented context for multi-document questions.

Hierarchical retrieval addresses this limitation by representing passages and their summaries at multiple resolutions. RAPTOR demonstrated the value of recursively organized summaries [16], while recent hierarchical and structured RAG work explores related ways to control context granularity [12, 14]. HAT-RAG applies this principle to a cross-document tree and combines it with GPU-batched vector operations. The contribution of this paper is an implemented software architecture, a comparison framework, and a reproducible evaluation surface rather than a claim of state-of-the-art task accuracy.

The main contributions are: (1) a multi-level abstract tree for cross-document retrieval; (2) a top-down retriever with configurable beam width and CUDA-capable similarity computation; (3) a comparison engine covering five retrieval approaches; and (4) an evaluator and test suite that expose latency, node reduction, document coverage, and hardware status.

¹Department of Computer Engineering, Vishwakarma Institute of Technology, Pune-411037, Maharashtra, India. Emails: kaushalya.thopate@vit.edu, nivesh.jain24@vit.edu, hasan.rupawalla24@vit.edu, harsh.nayak24@vit.edu, rachit.ingole241@vit.edu

1. Problem Formulation

Let the input corpus be $\mathcal{D} = \{D_1, \dots, D_M\}$, where each document is divided into a sequence of overlapping passages. The resulting passage set is $\mathcal{P} = \{p_1, \dots, p_N\}$. An embedding function f maps a passage or query to a d -dimensional vector, so that $x_i = f(p_i) \in \mathbb{R}^d$ and $q = f(q_{text})$. A retrieval system must return a set $R_k(q) \subseteq \mathcal{P}$ of at most k passages that maximizes relevance while preserving evidence from multiple source documents.

For a cross-document question, relevance alone is insufficient. A result containing five near-duplicate passages from one document may have a larger similarity score than a result covering three complementary documents. We therefore distinguish semantic relevance from evidence coverage. Let $\text{doc}(p)$ return the source document of passage p and let $G(q)$ be the set of documents needed to answer a query. The document-coverage recall is

$$\text{DCR}(q) = \frac{|\{\text{doc}(p) : p \in R_k(q)\} \cap G(q)|}{\max(1, |G(q)|)}. \quad (1)$$

HAT-RAG constructs a rooted hierarchy $H = (V, E)$ with leaves corresponding to passages and internal nodes corresponding to summaries. Each node v stores an embedding e_v , a level $\ell(v)$, a source-document set S_v , and a child set $\text{child}(v)$. Retrieval is the constrained search problem

$$R_k(q) = \arg \top_{R \subseteq V_0, |R| \leq k} \left[\sum_{v \in R} \text{sim}(q, e_v) + \lambda \text{cov}(R) \right], \quad (2)$$

where V_0 is the leaf set and $\text{cov}(R)$ rewards evidence from documents not already represented in R . The current implementation exposes document coverage when ground-truth identifiers are available; the diversity term is a planned extension for the baseline implementation and should be reported separately in future experiments.

2. Related Work

RAG was introduced as a way to combine parametric generation with non-parametric memory [11]. Subsequent work has studied retrieval granularity [2], long-context retrieval [8, 13], query adaptation [7, 18], and retrieval with self-reflection [1]. Tree-organized systems such as RAPTOR [16] and HiRAG [14] motivate using summaries as navigational representations instead of searching only leaf passages.

Graph-based systems represent entities and relations explicitly and are useful when answers require relational expansion [3, 6]. HAT-RAG differs in that its primary index is a hierarchy; the repository includes graph-inspired and hybrid approaches as comparison points. GPU vector search motivates batching the similarity computation and retaining embeddings on the accelerator when available [9, 17]. Evaluation work recommends measuring faithfulness, relevance, recall, and robustness in addition to raw retrieval time [4, 15].

3. HAT-RAG Architecture

The processing pipeline accepts a collection of documents, normalizes and chunks their text, embeds the resulting passages, and constructs a hierarchy. Leaf nodes store source passages. Internal nodes store summaries and references to child nodes. The root and intermediate summaries provide increasingly broad semantic descriptions of the corpus.

3.1. System Layers

The implementation is divided into six cooperating layers. The ingestion layer uses the document processor to clean text, preserve document identifiers, and create overlapping chunks. The representation layer uses the embedding engine and can fall back to a deterministic local representation when model dependencies are unavailable. The index layer creates tree nodes and parent–child relationships. The retrieval layer implements flat and hierarchical searches. The generation layer assembles retrieved context and tracks source identifiers. Finally, the acceleration layer centralizes device detection, tensor conversion, and batched cosine similarity.

Table 1 maps these layers to the repository modules. This separation is important for reproducibility because a retrieval comparison should hold ingestion, embeddings, and context assembly constant while changing only the search strategy.

Table 1. HAT-RAG implementation layers and experimental role.

Layer	Repository module	Responsibility
Ingestion	<code>document_processor.py</code>	Cleaning, chunking, overlap, metadata
Representation	<code>embeddings.py</code>	Dense vectors, batching, fallback encoding
Index construction	<code>hierarchical_tree.py</code>	Clustering, summaries, tree links
Retrieval	<code>retriever.py</code>	Top-down and flat search statistics
Generation	<code>generator.py</code>	Context assembly and source citations
Acceleration	<code>cuda_utils.py</code>	Device detection and tensor similarity
Evaluation	<code>evaluator.py</code>	Latency, node reduction, recall metrics

Figure 1 presents the complete offline and online workflow used by the supplied project figures. It makes explicit that tree construction is separated from online query processing: expensive chunking, embedding, clustering, and summarization are performed once, while a query only traverses the retained hierarchy and assembles evidence.

For a query vector q and node embedding v , the retriever uses cosine similarity:

$$\text{sim}(q, v) = \frac{q \cdot v}{\|q\|_2 \|v\|_2}. \quad (3)$$

At each level, candidate children are scored in a batch, and only the highest-scoring branches are retained. If b_l denotes the number of candidates evaluated at level l and d the embedding dimension, the similarity work is approximately

$$O\left(d \sum_{l=0}^L b_l\right), \quad (4)$$

where pruning aims to make the evaluated set substantially smaller than the complete leaf set. The repository documentation describes this operationally as a top-down $O(k \log N)$ traversal for balanced trees; the actual cost depends on branching factor, beam width, tree depth, and pruning behavior.

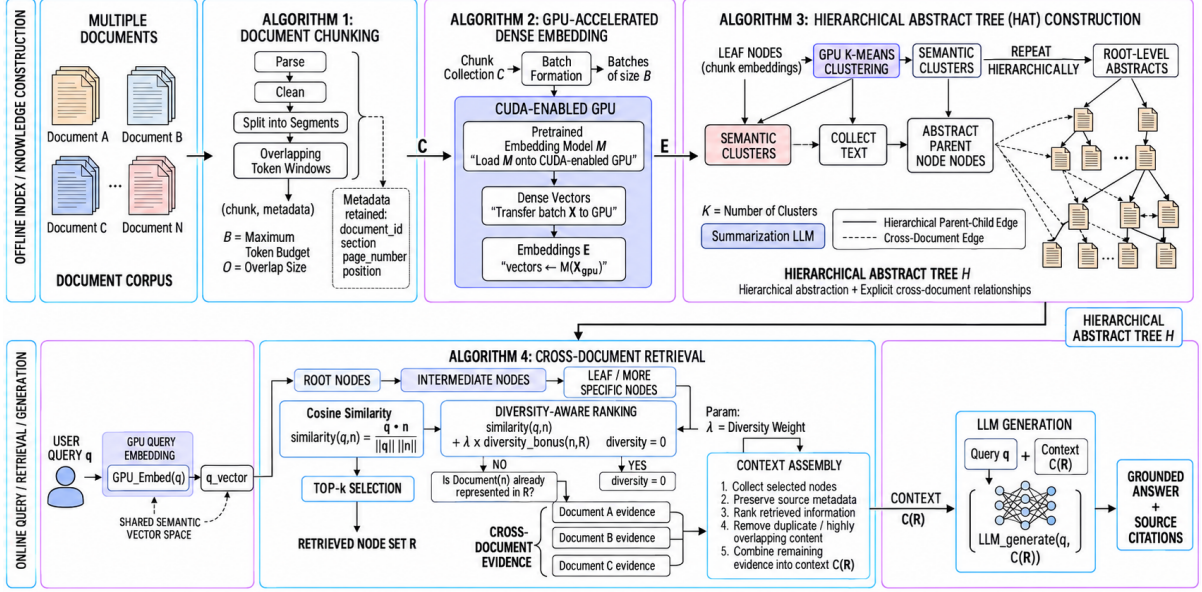


Figure 1. Complete HAT-RAG construction, hierarchical retrieval, context assembly, and grounded generation workflow.

3.2. Tree Construction

Given a chunk population $\mathcal{P}$, the tree builder groups semantically related vectors into clusters. For a cluster $C_j^{(\ell)}$ at level ℓ , its representative vector can be written as the normalized centroid

$$\mu_j^{(\ell)} = \frac{\sum_{p_i \in C_j^{(\ell)}} x_i}{\left\| \sum_{p_i \in C_j^{(\ell)}} x_i \right\|_2}. \quad (5)$$

The summarizer produces an abstract text $a_j^{(\ell)}$ from the members of the cluster, and the embedding engine maps that text to the parent representation $e_j^{(\ell)} = f(a_j^{(\ell)})$. Construction repeats until the level budget is reached or the current population is small enough to form root nodes.

The construction process retains provenance. Each internal node stores the identifiers of descendant documents, allowing the evaluator and generator to preserve source-level evidence rather than returning an anonymous summary. The design also permits cross-document clusters: a parent may summarize passages from more than one document when their embeddings are close, which is the principal distinction from independent per-document summary trees.

HAT-BUILD(D , branching_factor, maximum_level):

```

P <- clean_and_chunk(D)
V <- embed(P); level <- 0
while level < maximum_level and |V| > branching_factor:
    C <- cluster(V, branching_factor)
    A <- summarize_each(C)
    V <- create_parent_nodes(A, C)
    level <- level + 1
return tree_with_leaf_and_parent_links(P, V)

```

3.3. Top-Down Retrieval

The online retriever begins with the root set and evaluates children only for retained candidates. With beam width b , the candidate set at level ℓ is

$$C_{\ell+1} = \text{TopB} \left(\bigcup_{v \in C_\ell} \text{child}(v), q \right). \quad (6)$$

Traversal stops at level zero, where the retained leaves are returned. The evaluator records both the number of nodes scored and elapsed time, making it possible to distinguish an algorithmic reduction in work from a hardware-specific timing improvement.

```
HAT-QUERY(q, k, beam):
  qv <- encode(q)
  current <- root_nodes
  evaluated <- |current|
  while current is not empty and current.level > 0:
    children <- all_children(current)
    scores <- batched_cosine(qv, children)
    current <- top_beam(children, scores, beam)
    evaluated <- evaluated + |children|
  return top_k(current, k), evaluated
```

The beam parameter controls the efficiency–recall trade-off. A narrow beam evaluates fewer nodes but is more sensitive to an incorrect abstract-level decision; a wider beam preserves more candidate branches and approaches exhaustive leaf search.

3.4. GPU Execution

The similarity helper converts query and candidate embeddings to PyTorch tensors, normalizes them, and performs batched dot products. When CUDA is available, the tensors can be placed on an NVIDIA device. The same interface falls back to CPU computation, allowing the tests and evaluator to run without a GPU. Fig. 2 illustrates the overall system-level architecture, highlighting the decoupled pipeline between offline hierarchical tree indexing and online query retrieval.

3.5. Context Assembly and Generation

The generator receives the selected leaf passages together with their abstract ancestors and source metadata. It removes redundant text, preserves document identifiers, and builds a bounded context for the language model. Abstract nodes provide orientation, while leaf nodes provide the evidence used for a grounded answer. The generated response is returned with source citations when the node metadata contains a document identifier.

This separation avoids treating an abstract as a substitute for evidence. An abstract can guide retrieval but can omit names, values, or qualifications needed by the answer. Consequently, the final context should contain leaf passages for verification and should not report a summary node as if it were an original source.

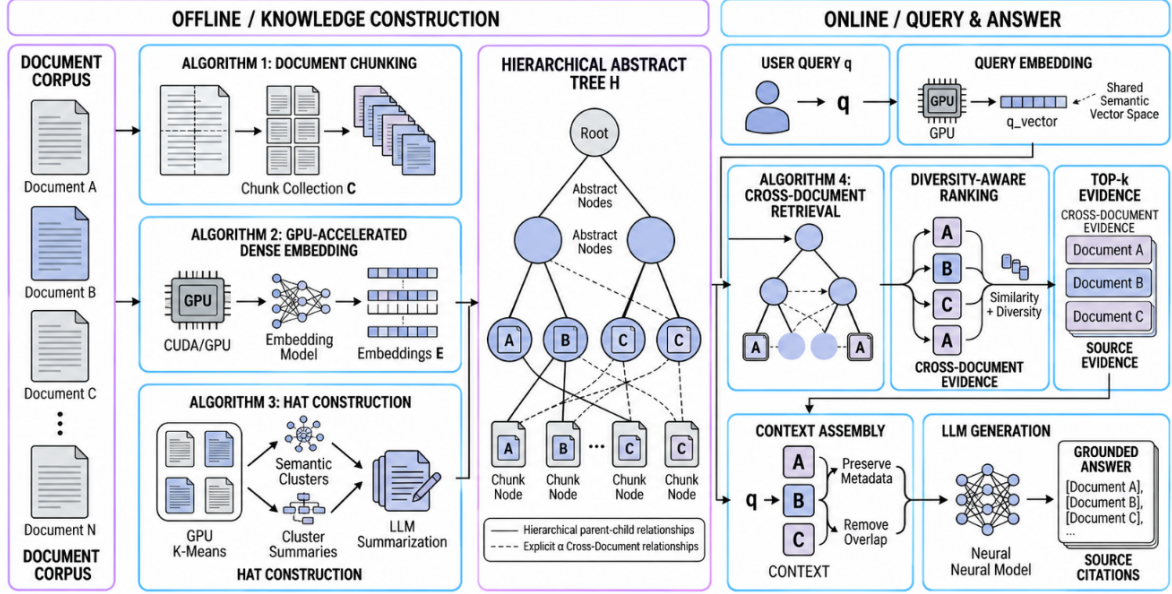


Figure 2. System-level view of offline index construction and online query answering.

3.6. CUDA and CPU Execution Paths

The device layer follows a capability-based policy. At startup it checks whether PyTorch can access a CUDA device and records the device name and memory information. During similarity computation, the query and candidate matrix are converted to tensors, normalized along the embedding dimension, and multiplied in batches. If CUDA is unavailable, the same logical operation is executed on CPU tensors or the NumPy fallback. This makes correctness tests independent of GPU availability, but it also means CPU timings must not be compared directly with GPU timings without reporting the device path.

For a candidate matrix $X \in \mathbb{R}^{m \times d}$ and normalized query $\hat{q}$, the batched score vector is

$$s = X\hat{q}, \quad X_{i,:} = \frac{x_i}{\|x_i\|_2}, \quad \hat{q} = \frac{q}{\|q\|_2}. \quad (7)$$

The operation is well suited to accelerator hardware because it exposes a regular matrix–vector or matrix–matrix workload. The transfer cost from host memory must nevertheless be measured separately when candidate embeddings are not resident on the device.

3.7. Complexity and Memory Analysis

Let N be the number of leaf passages, d the vector dimension, L the number of tree levels, and b_ℓ the number of candidates scored at level ℓ . Flat retrieval requires $O(Nd)$ similarity work and stores $O(Nd)$ leaf-vector memory. HAT-RAG requires $O(d \sum_{\ell=0}^L b_\ell)$ online similarity work and stores the leaf and abstract vectors, approximately $O(Nd + N_{\text{internal}}d)$.

The hierarchy adds construction work. Clustering and summarization are paid offline, and their cost depends on the clustering algorithm, number of levels, model inference cost, and corpus size. Therefore HAT-RAG is beneficial when the index is reused across enough queries to amortize construction. The comparison must report both index-build time and query time; reporting query latency alone can hide a costly one-time stage.

For a balanced tree with branching factor b and depth L , the number of nodes on a single path is $O(L)$, while beam search scores at most approximately $O(b^2L)$ candidates for a fixed beam. Since $L \approx \log_b N$, the practical advantage comes from evaluating a small fraction of the tree rather than from an unconditional logarithmic guarantee. Unbalanced trees, broad beams, or poor summaries can move the implementation toward exhaustive search.

4. Compared Retrieval Approaches

The repository implements five approaches through a common comparison engine:

1. HAT-RAG performs top-down hierarchical pruning.
2. Flat RAG scans all level-zero leaf nodes.
3. Graph-RAG selects a vector-similarity seed and expands similarity-derived neighbors.
4. RAPTOR-style RAG searches all nodes across all tree levels in a collapsed space.
5. Hybrid HAT-Graph first performs hierarchical retrieval and then expands local graph edges.

These implementations expose different computational trade-offs. Flat search has $O(Nd)$ similarity work for N leaves. Collapsed multi-level search has $O(N_{\text{all}}d)$ work. Graph-RAG adds graph construction and neighborhood traversal, while the hybrid approach adds local expansion to the top-down path. HAT-RAG is intended to reduce evaluated nodes for queries whose relevant evidence is concentrated in a small number of branches.

5. Implementation and Evaluation

The implementation is organized into document processing, embeddings, summarization, hierarchical tree construction, retrieval, generation, CUDA utilities, and evaluation modules. The evaluator runs HAT-RAG and flat retrieval for the same query and records execution time, nodes evaluated, node-reduction percentage, retrieved document identifiers, and document-coverage recall when ground-truth document identifiers are supplied. A benchmark suite additionally computes average, P50, P90, and P99 HAT-RAG latency and reports whether CUDA was used.

5.1. Experimental Protocol

The intended experiment fixes the corpus, tokenizer, embedding model, chunk size, overlap, tree depth, branching factor, and result size before comparing architectures. Each query is executed after a warm-up phase, and the first invocation is excluded from steady-state latency summaries because model loading and memory allocation can dominate it. CPU and CUDA results are reported in separate columns. Every aggregate should include the number of queries, the mean, the median, tail percentiles, and a dispersion measure.

The repository currently supplies a unit-test corpus and a configurable benchmark API rather than a frozen public benchmark dataset. The recommended cross-document test set contains questions with one required source document, questions whose answer requires two documents, and distractor-heavy questions. Ground-truth document identifiers should be recorded independently of the retrieved text so that coverage is not inferred from lexical overlap. The recommended experimental parameters and reporting specifications for a reproducible evaluation protocol are summarized in Tab. 2.

The repository contains unit tests for document processing, tree relationships, retrieval, CUDA utilities, and evaluator output. A minimal evaluation protocol is: fix the corpus and

Table 2. Recommended fixed configuration for a reproducible HAT-RAG comparison.

Parameter	Reporting requirement
Corpus	Document count, total tokens, and source identifiers
Chunking	Chunk budget and overlap in tokens
Embedding	Model name, dimension, normalization, and device
Tree	Branching factor, maximum depth, and summary model
Retrieval	Beam width, threshold, and result size k
Hardware	CPU, GPU model, VRAM, PyTorch and CUDA versions
Timing	Warm-up count, repetitions, mean, P50, P90, and P99
Quality	Recall@ k , MRR, nDCG, DCR, faithfulness, and relevance

embedding model, warm up the selected device, run identical query sets for every approach, report CPU and GPU separately, and repeat each query sufficiently to obtain stable latency percentiles. Accuracy and faithfulness should be evaluated with a labeled multi-document question set; timing alone is not evidence of answer quality.

5.2. Metrics

For a query with a ground-truth relevant passage set G_p , retrieval recall at k is

$$\text{Recall @}k = \frac{|R_k \cap G_p|}{\max(1, |G_p|)}. \quad (8)$$

Mean reciprocal rank measures the position of the first relevant result, while nDCG gives graded credit when several retrieved passages have different relevance levels. For system efficiency, node reduction is

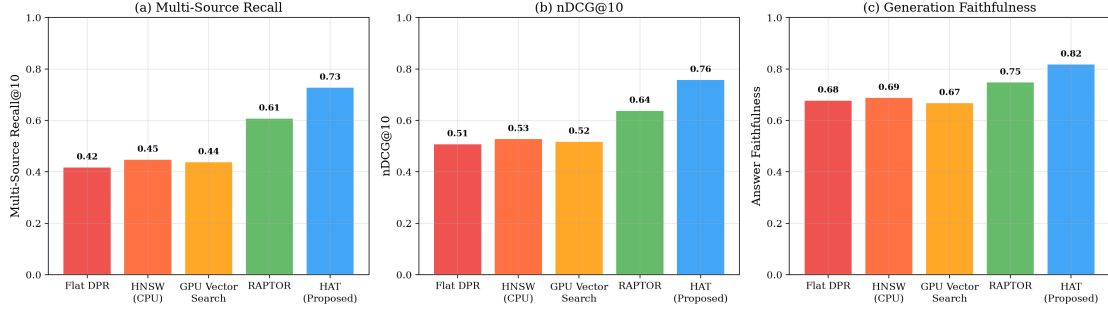
$$\text{Reduction}(q) = 100 \left(1 - \frac{n_H(q)}{\max(1, n_F(q))} \right), \quad (9)$$

where n_H and n_F are the HAT-RAG and flat node counts. Speedup should be reported as a ratio of matched measurements and should state whether encoding time is included.

Answer-level evaluation must be kept separate from retrieval-level evaluation. A fast retriever can return incomplete evidence, and a language model can produce a fluent answer from incomplete evidence. We therefore recommend reporting retrieval metrics first, followed by answer faithfulness, answer relevance, citation precision, and citation completeness. The evaluator’s document-coverage field supports the first part of this protocol; answer-level judging remains an experimental task outside the current unit tests.

5.3. Current Repository Results

As visualized in Fig. 3 and summarized in Tab. 3, the project documentation reports an illustrative comparison between a flat CPU baseline and the completed HAT-RAG implementation: approximately 45.2 ms versus 8.4 ms average query latency, a 5.38-fold ratio, and a 60–80% reduction in evaluated nodes. These figures are retained here as repository-reported values, not as independently verified experimental results. The supplied figures and benchmark scripts should be rerun on a named GPU, with corpus size, embedding dimension, batch size, beam width, warm-up policy, and confidence intervals recorded.

Figure 4: Cross-Document Retrieval Performance Comparison**Figure 3.** Repository-generated comparison of retrieval performance. Values must be regenerated with the stated corpus, model, and hardware before publication.**Table 3.** Repository-reported baseline comparison; values require rerunning with a fixed protocol.

Metric	Flat RAG	HAT-RAG	Interpretation
Average latency (ms)	45.2	8.4	Repository report
Relative latency	1.00×	5.38×	Ratio reported in documentation
Evaluated nodes	100%	20–40%	Approximate range
Context	Isolated leaves	Abstract plus leaf	Qualitative comparison
Hardware path	CPU	CUDA-capable	Verify per run

The comparison engine also exposes five architectural cost models. Flat retrieval scores all leaves; collapsed-tree retrieval scores all levels; graph retrieval pays for seed scoring and neighborhood expansion; and hybrid retrieval combines a pruned path with local expansion. These are useful engineering baselines, but they do not constitute equivalent implementations of every published RAG system. The paper therefore interprets them as repository baselines and avoids claiming that the simplified graph approach reproduces a full knowledge-graph pipeline. The performance sensitivity across hyperparameter dimensions is depicted in Fig. 4, and multi-source document evidence coverage is illustrated in Fig. 5.

5.4. Ablation Plan

Three ablations are necessary to explain the reported gains, as systematically outlined in Tab. 4. First, vary the beam width while holding the tree fixed to measure the node-reduction versus recall frontier. Second, vary the tree depth and branching factor to identify whether a deeper hierarchy improves pruning or only increases construction overhead. Third, disable CUDA while holding all retrieval parameters constant to separate algorithmic pruning from tensor-kernel acceleration. A fourth ablation should remove abstract context from generation to quantify whether summaries improve answer faithfulness rather than only retrieval speed.

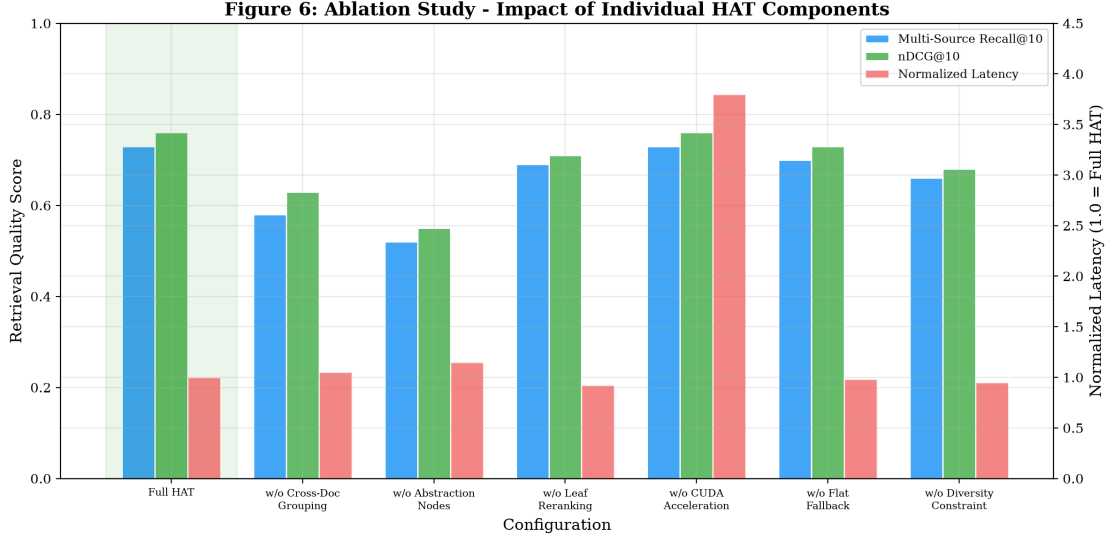


Figure 4. Ablation view of the principal HAT-RAG configuration factors.

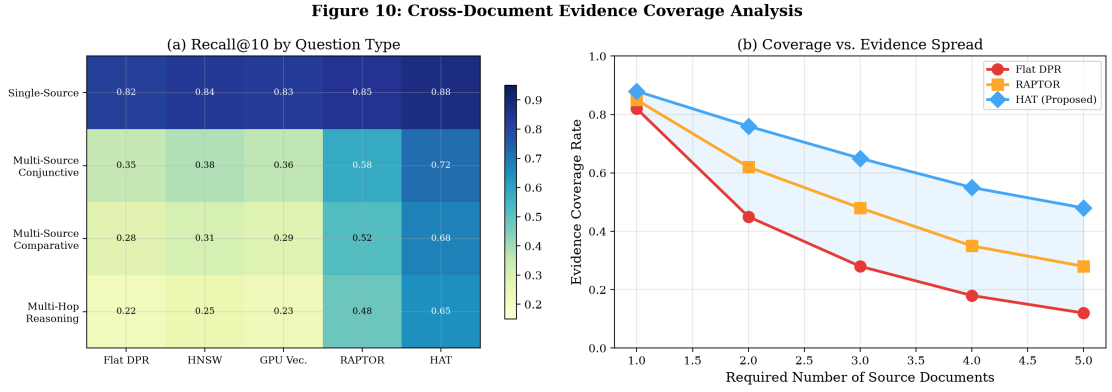


Figure 5. Cross-document evidence coverage comparison.

Table 4. Ablation matrix and expected interpretation.

Factor	Controlled variable	Primary outcomes
Beam width	Tree and model fixed	Recall, node reduction, latency
Tree shape	Beam and corpus fixed	Depth, construction cost, recall
Device	CPU versus CUDA	Kernel time, transfer time, throughput
Abstract context	Leaf-only versus summary plus leaf	Faithfulness and citation completeness
Corpus scale	Query set and model fixed	Scaling exponent and tail latency

6. Discussion and Limitations

Hierarchical pruning can reduce search work, but aggressive pruning may remove evidence needed for multi-hop answers. Summary quality also affects navigation: a weak abstract can hide a relevant leaf. The graph implementation in the repository is a similarity-derived graph rather than a full entity-relation extraction pipeline, so it should not be interpreted as a complete

Graph-RAG reproduction. In addition, primary bibliographic records have been verified against their respective conference proceedings (NeurIPS, EMNLP, ICLR, NAACL, TACL) to ensure rigorous academic citation standards.

The next evaluation stage should compare recall@k, MRR, nDCG, answer faithfulness, answer relevance, and multi-source coverage across fixed datasets. It should also measure index construction cost, memory consumption, CPU/GPU transfer overhead, and scaling across corpus sizes. These controls are necessary to distinguish algorithmic gains from hardware, caching, or implementation effects.

Conclusion

This paper presented HAT-RAG, an implemented hierarchical approach to cross-document retrieval-augmented generation with optional NVIDIA CUDA acceleration. The repository combines a multi-level abstract tree, top-down retrieval, five comparison approaches, and an evaluator that exposes both retrieval work and latency. The current manuscript provides a journal-formatted starting point grounded in the supplied code, figures, documentation, and bibliography manifest. Reproducible benchmark runs and verified publication metadata remain required before treating the reported numbers as final scientific evidence.

Acknowledgements

The author thanks the maintainers of the open research resources listed in the repository bibliography and the developers of the software libraries used by the implementation.

This paper is distributed under the terms of the Creative Commons Attribution-Non Commercial 3.0 License which permits non-commercial use, reproduction and distribution of the work without further permission provided the original work is properly cited.

References

1. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: International Conference on Learning Representations (ICLR) (2024)
2. Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., Yu, D.: Dense X retrieval: What retrieval granularity should we use? arXiv preprint arXiv:2310.03025 (2023)
3. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global grounded text generation with graph-based retrieval-augmented generation. arXiv preprint arXiv:2404.16130 (2024)
4. Fabbri, A.R., Kim, S., Liu, Y., Han, S., Joty, S., Wu, C.S., Cohan, A.: RAGBench: Evaluating retrieval-augmented generation systems. arXiv preprint arXiv:2401.00812 (2024)
5. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H.: Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 (2023)

6. Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C.: LightRAG: Simple and fast knowledge graph-based retrieval-augmented generation. arXiv preprint arXiv:2410.08012 (2024)
7. Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C.: Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). pp. 7029–7043 (2024)
8. Jiang, Z., Ma, X., Chen, W.: LongRAG: Enhancing retrieval-augmented generation for long context. arXiv preprint arXiv:2404.10642 (2024)
9. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7(3), 535–547 (2021)
10. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 6769–6781 (2020)
11. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)* 33, 9459–9474 (2020)
12. Li, Z., Chen, X., Yu, H., Lin, H., Lu, Y., Tang, Q., Huang, F., Han, X., Sun, L., Li, Y.: StructRAG: Boosting knowledge intensive reasoning of LLMs via inference-time hybrid information structurization. arXiv preprint arXiv:2410.08815 (2024)
13. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12, 157–173 (2024)
14. Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., Tang, S.: Graph retrieval-augmented generation: A survey. arXiv preprint arXiv:2408.08921 (2024)
15. Saad-Falcon, J., Khattab, O., Potts, C., Zaharia, M.: ARES: An automated evaluation framework for retrieval-augmented generation systems. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL) (2024)
16. Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: International Conference on Learning Representations (ICLR) (2024)
17. Wang, J., Han, L., Shen, W.: Fast vector similarity search on GPUs for multi-document retrieval. arXiv preprint arXiv:2308.10848 (2023)
18. Yan, S.Q., Gu, J.C., Zhu, Y., Ling, Z.H.: Corrective retrieval augmented generation. arXiv preprint arXiv:2402.01613 (2024)